

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Циклы	2
Цикл while	2
Операторы break и continue	3
Цикл for	5
Функция zip	7
Инструкция else в циклах for и while	7
Функция range	8
Функция enumerate	9
Генераторы последовательностей	9

Циклы

Одной из наиболее часто используемых конструкций почти во всех языках программирования являются циклы. Они позволяют выполнять повторяющиеся действия, изменяя или дополняя их на каждом этапе. В python существует 2 вида циклов – цикл **while** и цикл **for**.

У начинающих часто возникает вопрос, когда нужно использовать первый, а когда второй цикл? Ответ на этот вопрос нужно запомнить: мы используем цикл **while** в тех случаях, когда **не знаем, сколько нам понадобится повторений – итераций**, если же мы **знаем или можем подсчитать** это количество в коде, используем цикл **for**.

Сразу отмечу, что большей части случаев это будет цикл **for**.

Цикл while

На прошлом уроке мы познакомились с условием **if**, тело которого выполнялось **только один раз**, в случае когда результат **True** был в условии.

Но мы можем выполнять тело условия не один раз, а до тех пор, пока это условие не изменится на **False** – это и будет называться циклом **while**, для его создания используется оператор **while** («пока»). За этим словом точно так же идёт условие, результат которого будет приведён к булеву типу, и мы получим **True** или **False**, заканчивается это выражение так же двоеточием. За двоеточием идёт блок кода с отступом (тело цикла). Этот блок продолжит выполняться, пока результат выражения остается равным **True**.

Запись цикла выглядит так:

```
while [условие]:  
    [тело цикла]
```

Как видите, для создания цикла нам нужны следующие:

- Ключевое слово **while**
- Условие цикла
- Тело цикла, где будет происходить изменение условия

Пример выполнения обратного отсчёта с помощью цикла **while**

```
In [1]: counter = 10  
...:  
...: while counter > 0:  
...:     print(counter, end=' ')  
...:     counter -= 1  
...:  
10 9 8 7 6 5 4 3 2 1
```

Если изменения условия не происходит, то мы получим **бесконечный цикл**.

```
In [2]: counter = 10
...:
...: while counter > 0:
...:     print(counter, end=' ')
...:     # counter -= 1
...:
10 10 10 10 10 10 10 10 10 10 10 10 10
```

Стоит отметить, что именно через цикл **while** в python реализуется бесконечный цикл, иногда им необходимо пользоваться для выполнения ряда задач.

Операторы **break** и **continue**

Когда нам нужно прервать цикл при выполнении какого-то условия, используется оператор **break**.

```
In [3]: counter = 10
...:
...: while counter > 0:
...:     if counter == 5:
...:         break
...:     print(counter, end=' ')
...:     counter -= 1
...:
...: print('\nВышли из цикла!')
10 9 8 7 6
Вышли из цикла!
```

Мы остановили выполнение цикла, но код программы продолжил выполняться.

```
In [4]: counter = 10
...:
...: while counter > 0:
...:     counter -= 1
...:
...:     if counter == 5:
...:         continue
...:
...:     print(counter, end=' ')
...:
...: print('\nВышли из цикла!')
9 8 7 6 4 3 2 1 0
Вышли из цикла!
```

Инструкция **continue** имеет схожую логику, отличие в том, что она не прервёт весь цикл, а пропустит только ту итерацию, на которой вызвана и цикл продолжится со следующей итерации.

Пример кода, с выводом аналогичным предыдущему примеру, только с пропущенной цифрой 5.

Мы можем использовать любые операторы нужное количество раз

```
In [5]: while True:
...:     answer = int(input('Введите число от 0 до 10\n'))
...:
...:     if answer == 10:
...:         print('Меткий стрелок попадает в десятку')
...:         continue # инструкция перехода к следующему шагу цикла
...:     elif 7 <= answer < 10:
...:         print('Хорошо стреляешь, но есть куда стремиться')
...:     elif answer > 10:
...:         print('Робин Гуд нашёлся! :-)')
...:     else:
...:         print('Мазила!')
...:         break
...:
Введите число от 0 до 10
100
Робин Гуд нашёлся! :-)
Введите число от 0 до 10
7
Хорошо стреляешь, но есть куда стремиться
Введите число от 0 до 10
10
Меткий стрелок попадает в десятку
Введите число от 0 до 10
6
Мазила!
```


Цикл for

Цикл for используется гораздо чаще и является более гибким. Конструкция цикла выглядит следующим образом

```
for [переменная/ые] in [последовательность]:
    [тело цикла]
```

После ключевого слова **for** мы указываем название переменной, которая будет создаваться на каждой итерации цикла, оно может быть любым, но нужно иметь в виду один момент – в случаях, когда мы хотим показать, что мы не будем использовать переменную в теле цикла, именовать её стоит нижним подчёркиванием.

```
for _ in [последовательность]:
    [тело цикла]
```

Подобный синтаксис делает код понятнее. Если же мы будем использовать создаваемую переменную в теле цикла, ей нужно придумать понятное имя, соответствующее PEP8.

После имени переменной идёт ключевое слово **in** и итерируемая последовательность, элементы которой будут перебираться в цикле:

```
In [1]: for letter in ['G', 'O', 'O', 'D', '!']:
...:     print(letter, end='-')
...:
G-O-O-D-!-
```

В примере выше мы перебирали элементы списка, но как мы помним, строка тоже является своего рода списком, значит и её так же можно перебрать в цикле:

```
In [2]: for letter in 'GOOD!':
...:     print(letter, end='-')
...:
G-O-O-D-!-
```

Довольно быстро вы запомните, какие объекты можно перебирать в цикле – итерировать, а какие нет. Главным критерием перебора является наличие у объекта служебного метода `__iter__()`. Функция `dir()` покажет вам все методы объекта и поможет понять, получится перебрать в цикле объект или нет

```
In [3]: print(dir('string'))
['_add_', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getnew
args__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__
', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__',
'__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__',
```

Как видите, у строки он есть, значит, её можно итерировать в цикле. А вот у чисел подобного метода не будет, поэтому при попытке перебрать их в цикле, мы получим ошибку.

Ещё один важный момент – цикл в python предоставляет возможность создавать сразу **несколько** переменных в условии цикла, главное, чтобы каждой переменной соответствовало какое-то значение:

```
In [4]: tuples = ('ab', 'bc', 'cd')
...: for var_1, var_2 in tuples:
...:     print(var_1, var_2)
...:
a b
b c
c d
```

В примере выше у нас есть кортеж строк, который мы перебираем в цикле. Так как в каждом элементе этого кортежа есть строка из двух символов, для каждого из этих символов мы можем создать свою переменную.

Если же какой-то метод объекта возвращает разу несколько значений, например, метод `.items()` у словаря, мы можем создать и для ключа, и для значения:

```
In [5]: results_dict = {80: 'Apache', 8080: 'Flask', 22: 'SSH'}
...: for port, service in results_dict.items():
...:     print("Порт:", port, '\tзапущенный сервис:', service)
...:
Порт: 80      запущенный сервис: Apache
Порт: 8080    запущенный сервис: Flask
Порт: 22      запущенный сервис: SSH
```

Внутри тела цикла мы можем использовать те же конструкции, что и вне его – объявления переменных, создание условий и других циклов.

Функция zip

В случае, когда нам нужно перебрать сразу несколько разных итерируемых последовательностей, используется функция `zip`, которая на каждой итерации цикла будет возвращать нам пару – первый элемент первого итерируемого объекта и первый элемент второго итерируемого объекта:

```
In [5]: for var_1, var_2 in zip('abcdef', '123456'):
...:     print(var_1, var_2, sep=' - ')
...:
a - 1
b - 2
c - 3
d - 4
e - 5
f - 6
```

При несоответствии количества в итерируемых последовательностях мы получим минимально возможное количество пар:

```
In [6]: for var_1, var_2 in zip('abcdefghij', '123456'):
...:     print(var_1, var_2, sep=' - ')
...:
a - 1
b - 2
c - 3
d - 4
e - 5
f - 6
```

Инструкция else в циклах for и while

Ещё одной отличительной особенностью циклов `for` и `while` в python является наличие блока `else`. Главная его задача - выполнить код внутри себя в том случае, если мы прошли цикл до конца, то есть оператор `break` не был вызван.

Сравните два примера:

```
In [1]: for elem in range(4):
...:     print(elem)
...:     else:
...:         print('Блок else выполнен!')
...:     print('Вышли из цикла!')
...:
0
1
2
3
Блок else выполнен!
Вышли из цикла!
```

В первом случае мы прошли цикл до конца без прерывания и блок `else` выполнен после окончания цикла.


```
In [2]: for elem in range(4):
...:     print(elem)
...:     if elem == 2:
...:         break
...: else:
...:     print('Блок else выполнен!')
...: print('Вышли из цикла!')
```

0
1
2
Вышли из цикла!

Во втором примере после выполнения инструкции `break` мы просто вышли из цикла. Код из блока `else` не выполнялся.

Функция range

Функция `range` представляет собой простейший генератор последовательности чисел. В качестве аргументов функция принимает 3 числа:

1. начальное значение диапазона (**включительно**) (по умолчанию 0)
2. конечное значение (**не включительно**)
3. шаг (по умолчанию 1).

То есть если нам нужно получить последовательность чисел от 1 до 5 с шагом 1, условие цикла будет выглядеть следующим образом:

```
In [1]: for i in range(0, 5, 1):
...:     print(i)
...:
```

0
1
2
3
4

```
In [2]: for i in range(5):
...:     print(i)
...:
```

0
1
2
3
4

Так как начальное значение по умолчанию 0 и шаг тоже имеет значение по умолчанию 1, мы можем их не указывать:

Чтобы проитерировать последовательность в обратном порядке, нужно соблюсти два очевидных условия:

- 1) Начальное число должно быть больше конечного
- 2) Шаг должен быть отрицательным

```
In [3]: for i in range(5, -3, -3):
...:     print(i)
...:
```

5
2
-1

Обратите внимание вот на какой момент: для того, чтобы в цикле получить значение из последовательности – будь то список, кортеж, словарь или любой другой объект, используйте для перебора сам объект, а не его индексы

Неправильно

```
In [1]: users_list = ['User', 'Admin', 'guest', 'root']
...:
...: for num in range(len(users_list)):
...:     print(users_list[num])
...:
User
Admin
guest
root
```

Правильно

```
In [2]: users_list = ['User', 'Admin', 'guest', 'root']
...:
...: for target in users_list:
...:     print(target)
...:
User
Admin
guest
root
```

Функция enumerate

Мы можем также получить порядковые номера элементов последовательности с помощью функции [enumerate](#). Первым аргументом она принимает последовательность, вторым – индекс начала:

```
In [1]: team = ("Трус", "Балбес", "Бывалый")
...: for index, value in enumerate(team, 1):
...:     print(index, value)
...:
1 Трус
2 Балбес
3 Бывалый
```

Генераторы последовательностей

С помощью цикла и функции [range](#) мы можем создавать списки необходимых последовательностей.

```
In [1]: output_list = []
...: for elem in range(5, 22, 3):
...:     output_list.append(elem + 2)
...:
...: print(output_list)
[7, 10, 13, 16, 19, 22]
```

Аналогичный результат можно получить в 2 строки, используя так называемый «генератор списков».

```
In [2]: output_list = [elem + 2 for elem in range(5, 22, 3)]
...: print(output_list)
...:
[7, 10, 13, 16, 19, 22]
```

Воспринимать его нужно следующим образом: для каждого элемента последовательности от 5 до 22 не включительно с шагом 3 мы добавляем в список элемент, прибавляя к нему 2. То есть читаем это выражение от середины направо и возвращаемся в начало.

Подобная конструкция поддерживает и условия:

```
In [3]: output_list = [elem + 2 for elem in range(5, 22, 3) if elem > 11]
...: print(output_list)
...:
[16, 19, 22]
```

Мы добавляем в список только те значения `elem`, которые больше 11, прибавляя к ним 2.

Аналогичным образом мы можем создавать и словари. Отличие будет только в том, что нам придётся указать не один объект слева, а 2 – ключ и значение и в скобках, которые соответствуют этому типу данных

```
In [4]: letters_list = ['Аз', 'Буки', 'Веди']
...:
...: output_dict = {key: val for key, val in enumerate(letters_list, 1)}
...:
...: print(output_dict)
{1: 'Аз', 2: 'Буки', 3: 'Веди'}
```